www.Syskriti.in

SQL 100

A Complete Beginner's Guide to Structured Query Language

Why Databases Exist

You might wonder: why not just use Excel, Word, or PDFs to store data? Flat files can hold data, but databases solve four major problems that flat files can't.

1. Better Structure + Efficiency

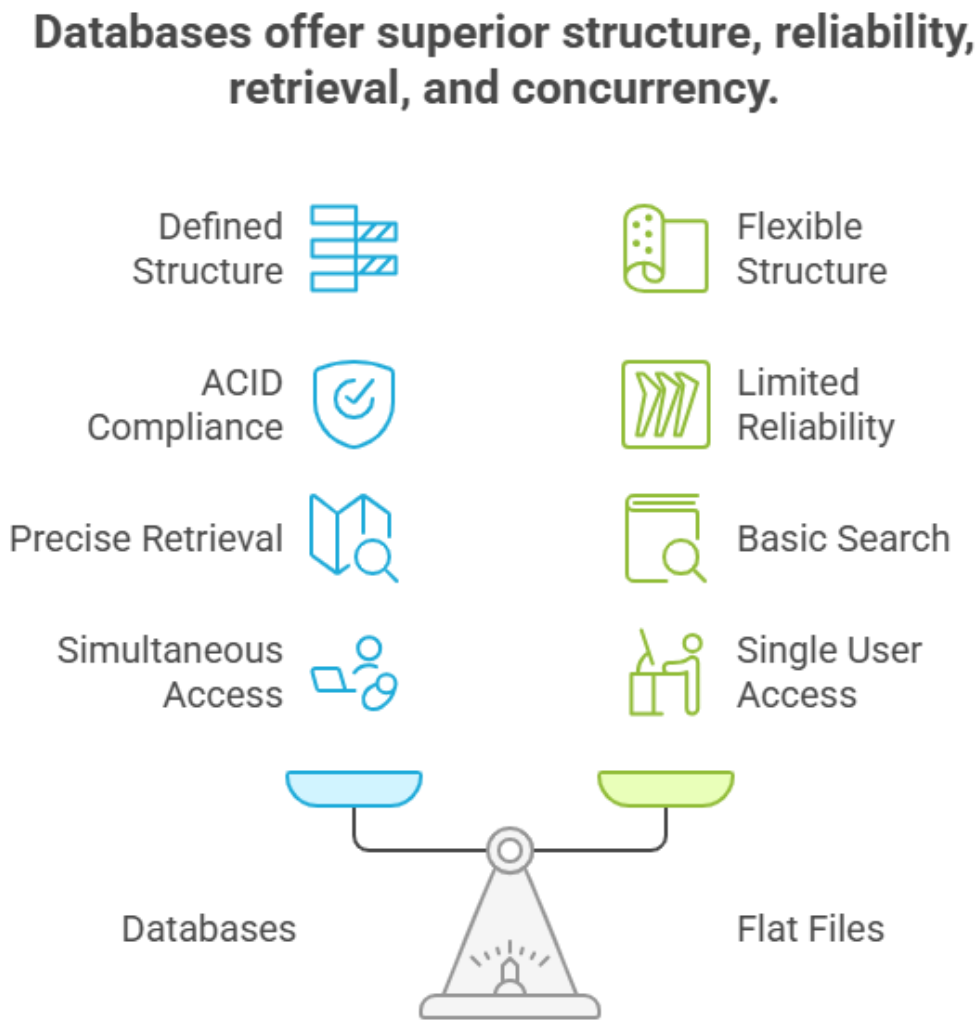
In a database, you define the structure first (called a schema), then store data inside that structure. In Word or Excel, you can type anything anywhere at any time — which sounds flexible, but means less control and more room for messy, inconsistent data.

2. Reliability via ACID

Databases follow the ACID principles — a set of guarantees that keep your data safe and correct:

- **Atomicity:** A transaction is "all or nothing." If any part of an operation fails, the entire thing is rolled back — no half-done updates.
- **Consistency:** The database stays in a valid, coherent state after every operation. Rules you've defined (constraints, data types) are always enforced.
- **Isolation:** When multiple transactions run at the same time, each one behaves as if it's the only one running. They don't interfere with each other or produce unexpected results.
- **Durability:** Once a change is saved (committed), it persists even if the system crashes or loses power.

@copyright : Syskriti Pvt Ltd 2026.



3. Powerful Retrieval

Flat files rely mostly on Ctrl+F. Databases let you retrieve data using precise, complex logic — filtering across columns, combining conditions, and pulling exactly what you need from millions of records.

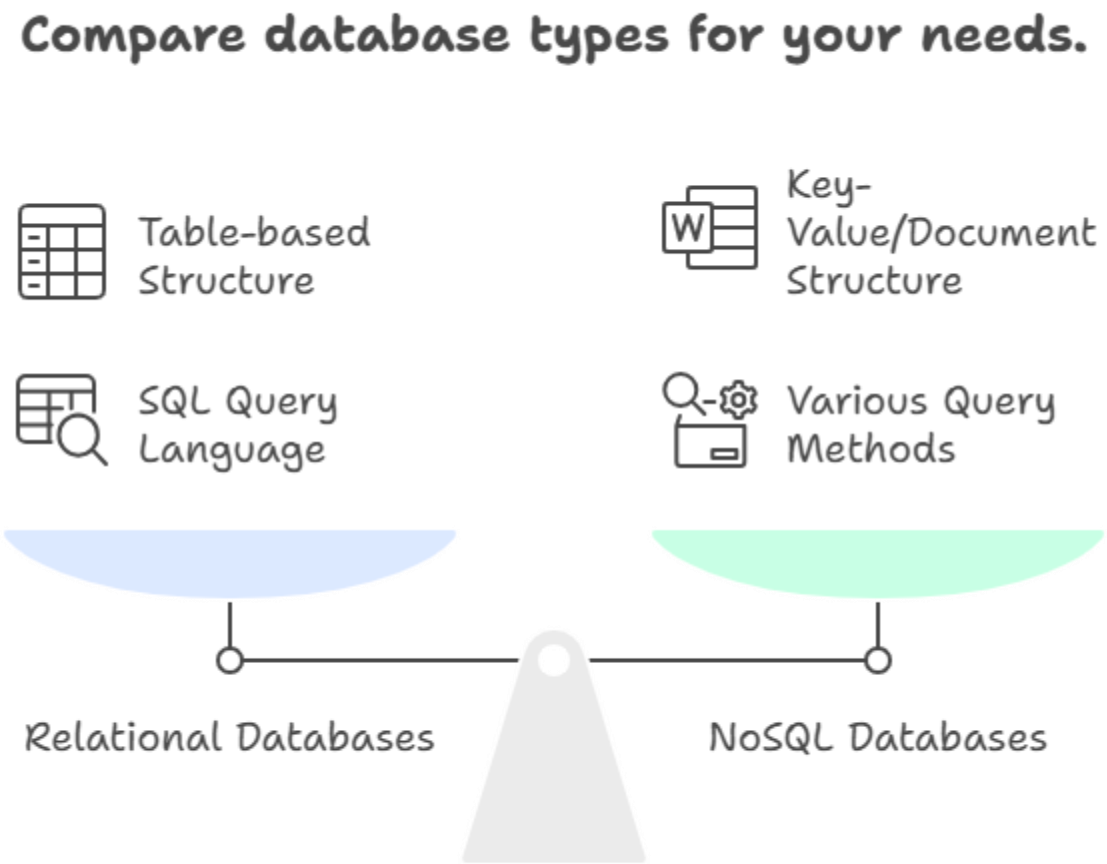
4. Concurrency

Many users can read from and write to a database at the same time. The database manages consistency behind the scenes so nobody's changes overwrite or corrupt someone else's work. This is where Isolation (the I in ACID) does its job.

@copyright : Syskriti Pvt Ltd 2026.

Relational vs. NoSQL

- **Relational databases** store data in tables (rows + columns). SQL is the language designed for these.
- **NoSQL databases** (short for “Not Only SQL”) store data in flexible formats like key-value pairs, documents, or graphs. Think of them as closer to dictionaries or JSON files. They complement relational databases rather than replace them.



@copyright : Syskriti Pvt Ltd 2026.

What SQL Is

SQL = Structured Query Language. It's a standard language used to interact with relational databases, regardless of vendor (Oracle, Microsoft SQL Server, PostgreSQL, IBM Db2, etc.). Without SQL, analysts would need to learn a different language for each database system.

SQL Sub-Languages

Even though analysts spend most of their time using SELECT, SQL is divided into several categories based on what the commands do:

1) **DDL — Data Definition Language** (build the structure)

Defines the "frame" before you put data in it: CREATE, ALTER, DROP.

2) **DML — Data Manipulation Language** (put and change the data)

Works with the data itself: INSERT, UPDATE, DELETE.

3) **DCL — Data Control Language** (who can access what)

Manages permissions — something nearly impossible with flat files: GRANT, REVOKE.

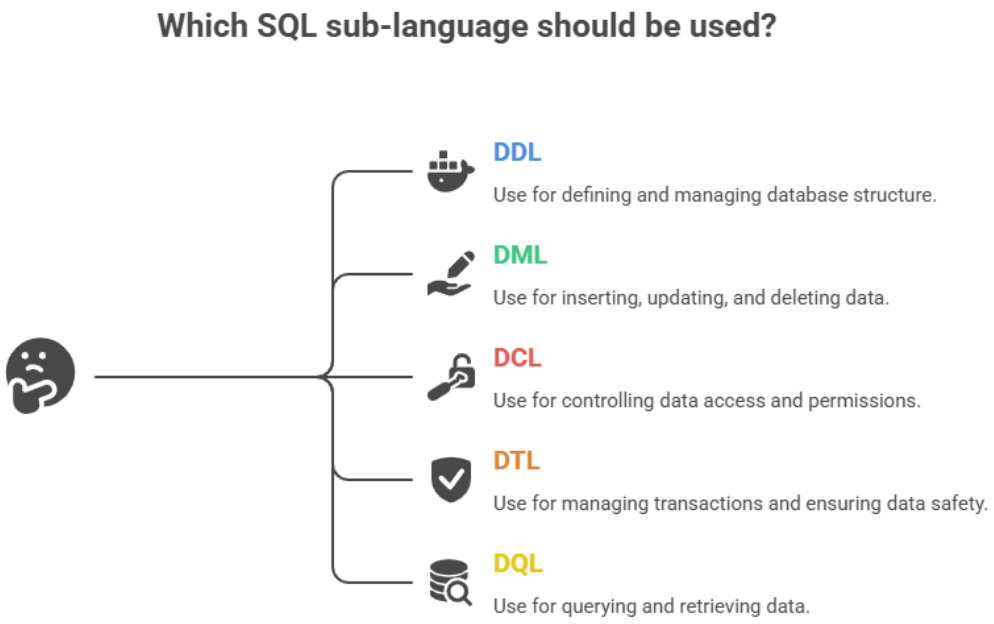
4) **DTL — Transaction Control** (keep operations safe)

Ensures operations complete safely: COMMIT, ROLLBACK.

5) **DQL — Data Query Language** (the analyst's main tool)

This is the querying part — and it boils down to one powerful command: SELECT.

@copyright : Syskriti Pvt Ltd 2026.



How Data Looks in a Relational Database

Data is stored in tables. Here's the anatomy:

- **Columns** = Fields or Attributes (what kind of information is stored)
- **Rows** = Records or Observations (each row is one complete "event")
- **Cell** = A single value at the intersection of a row and a column

A database contains many tables. For example, a business might have tables for Orders, Customers, Suppliers, and Finance.

@copyright : Syskriti Pvt Ltd 2026.

Relational Database Structure and Schema

Example: An Orders Table

OrderID	Date	ProductID	SupplierID	Quantity	Price
1001	2024-01-22	P45	S12	3	$899
1002	2024-01-23	P12	S07	10	$24

Schema: What Defines a Table's Structure

When you create a table, the schema defines the data type (number, text, date), the length (e.g., text up to 50 characters), and any constraints — rules the data must follow.

Common constraints include:

- **UNIQUE:** No two rows can have the same value in that column.
- **NOT NULL:** A value must exist — it can't be left blank.

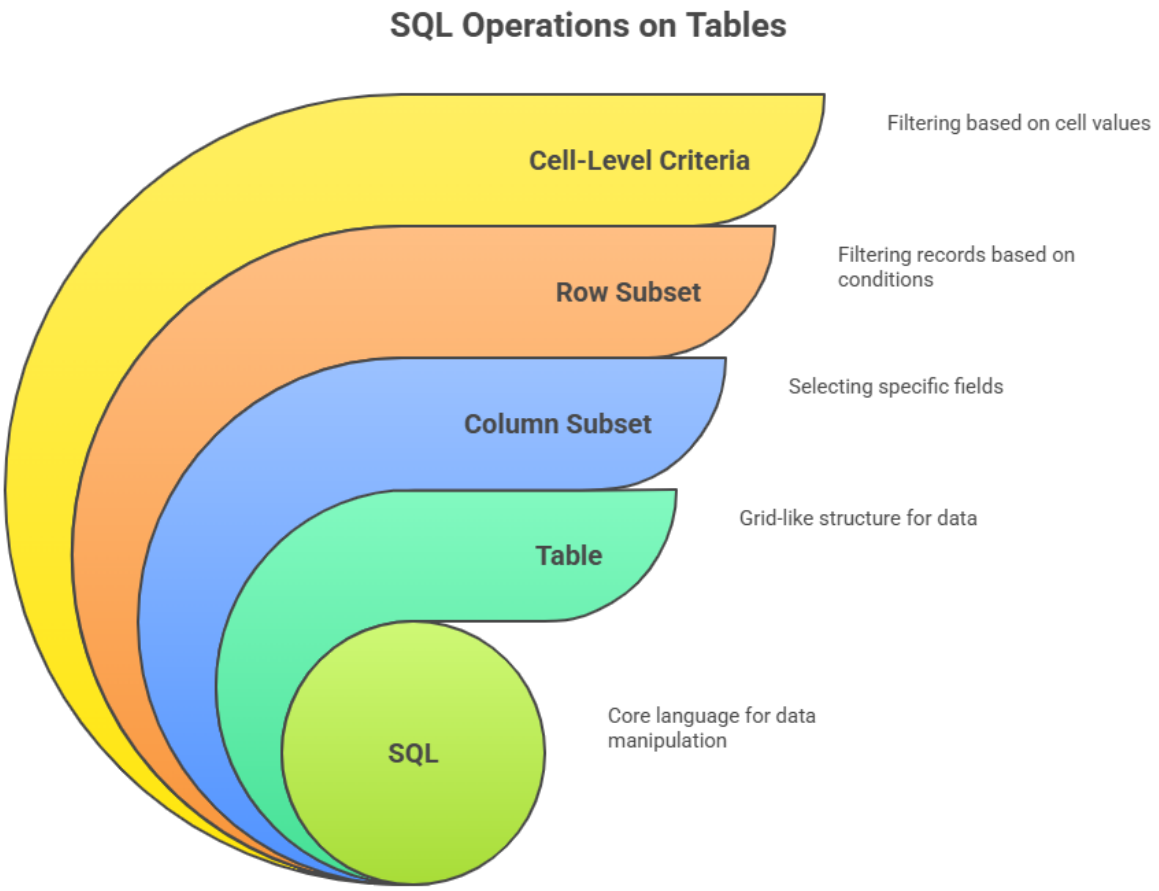
@copyright : Syskriti Pvt Ltd 2026.

The Core Idea: SQL as Algebra on a Table

Think of your table like a grid. SQL lets you do two fundamental operations:

1) **Pick your columns (SELECT):** Choose which fields you want to see.
2) **Pick your rows (WHERE):** Filter which records match your conditions.

That's it. Once you see SQL this way, every query becomes predictable: you're always choosing columns and filtering rows.



@copyright : Syskriti Pvt Ltd 2026.

The WHERE Clause: Two Types of Operators

The WHERE clause is where your filtering logic lives. It uses two types of operators working together:

Comparison Operators (testing values)

These test individual values within each row to see if they meet a condition:

- = (equals)
- > < >= <= (greater/less than)
- IN / NOT IN (matches any value in a list)
- BETWEEN (within a range)

Logical Operators (combining conditions)

These combine multiple comparison tests into a single filtering rule:

- **AND:** Both conditions must be true.
- **OR:** At least one condition must be true.
- **NOT:** Reverses the result of a condition.

How they work together: The database evaluates each row one by one. For every row, it checks the comparison operators to test individual values, then uses logical operators to combine those tests into a final yes/no decision on whether to include that row.

Example: *"Orders where the product is 'Laptop' AND the date is Jan 22, 2024"*

What Happens When You Run a Query

1) You type SQL in an IDE or database tool.
2) Your query is sent to a SQL engine.
3) The engine executes it against the database and returns results.
4) The result set you see is temporary (held in memory), not permanently stored.

If you need the results later, you either run the query again or save the query itself as a view. Note: a view doesn't store data — it saves the query definition and re-runs it each time you access it, so results are always up to date.

@copyright : Syskriti Pvt Ltd 2026.

Why Results Look "Random"

Database rows are not stored in any guaranteed order. Running the same query twice might return rows in a different sequence. To control the display order, use:

```
ORDER BY column_name;
```

This affects presentation only, not how data is stored.

LIMIT: Preview a Small Sample

When a table has thousands or millions of rows, you often want a quick look at just a few:

```
LIMIT 100;
```

This returns only the first 100 rows, making it perfect for exploring unfamiliar data.

Quick Recap

Here's what you've learned:

- **Why databases exist:** structure, ACID reliability, powerful retrieval, concurrency.
- **What SQL is:** a standard language with sub-categories (DDL, DML, DCL, DTL, DQL).
- **How relational data is stored:** tables made of fields, records, and cells.
- **The core SQL pattern:** SELECT (columns) + FROM (table) + WHERE (row filter).
- **Filtering logic:** comparison operators test values; logical operators combine tests.
- **ORDER BY and LIMIT:** essential tools for exploring and presenting data.

What to Do Next

Memorize this query skeleton:

```
SELECT ...  FROM ...  WHERE ...  ORDER BY ...  LIMIT ...
```

Practice "thinking in tables" by asking yourself these three questions before writing any query:

1) Which columns do I need?
2) Which rows match my condition?
3) Do I need sorting, or just a sample?

Once this becomes second nature, you're ready for SQL 200.

@copyright : Syskriti Pvt Ltd 2026.